

Supplementary Material: Disentangling Parser and Data Effects in Tool-Using LLM Fine-Tuning

Anonymous Author(s)
Paper under double-blind review

S1. Parser source

S1.1 V1 parser (single-pass)

```
TOOL_CALL_PATTERN = re.compile(
    r"<tool_call>\s*(\{.*?\})\s*</tool_call>",
    re.DOTALL,
)
# On malformed input: no match; server returns raw text as final answer.
```

S1.2 V2 parser (two-pass, used in E4 and V2)

```
TOOL_CALL_CLOSED = re.compile(
    r"<tool_call>\s*(\{.*?\})\s*</tool_call>",
    re.DOTALL,
)
TOOL_CALL_OPEN = re.compile(
    r"<tool_call>\s*(\{.*?\})",
    re.DOTALL,
)
# Pass 1: strict. Pass 2 (fallback): first JSON after any <tool_call>.
# Known limitation: Pass 2 recovers only the first call in a malformed
# multi-call output and may truncate nested arguments.
```

S2. Training trajectory counts

Condition	Trajectories	Source file
V1	35	training_data.jsonl
V2	46	training_data_augmented.jsonl

The 11 additional trajectories address: word_count→calculator (3), weather→calculator (3), three-tool synthesis (2), division-by-zero handling (2), false-premise rejection (1).

S3. Environment

Field	Value
Python	3.11.9
torch (local eval)	2.13.0+cpu
transformers	5.15.1
peft	0.20.0
accelerate	1.14.0
Training GPU	1× NVIDIA T4 (15 GB, Colab)
Inference GPU	1× NVIDIA T4 (15 GB, Colab)
CUDA / driver	not recorded (gap)
Training seed	not set
Eval seed	not set

S4. Artifact mapping

Artifact	File
23-task benchmark	eval/tasks.py
Scoring harness	eval/harness.py
V1 training script	training/colab_train_7b.py
V2 training script	training/colab_train_7b_v2.py
V1 serving code	serving/colab_server.py
E4 serving code	serving/colab_server_e4.py
V2 serving code	serving/colab_server_v2.py
Base results	results/eval_results_base.json
V1 results	results/eval_results_v1.json
E4 results	results/eval_results_e4.json
V2 results	results/eval_results_v2.json
Summary	results/experiment_summary.json